GitHub Copilotで
既存リポジトリを段階的に改善する

テスト・リンター・CI/CDをCopilotで導入する実践手順

© 2026

既存コードは4ステップで安全に改善できる

1. テストの導入 — 既存コードを変えずに品質を担保
2. リンターの導入 — コードスタイルの自動統一
3. CI/CDの構築 — 変更の検証を自動化
4. Copilot指示書の作成 — AIにプロジェクト文脈を共有

© 2026

テストもリンターもないコードは変更が怖い

```
# app.py - FlaskベースのタスクAPI
from flask import Flask, request, jsonify
import sqlite3
import os   # ← 未使用のimport

@app.route('/tasks', methods=['GET'])
def get_tasks():
    conn = sqlite3.connect('tasks.db')  # テストなし、CIなし
```

テストなし / リンターなし / CI/CDなし → **変更するたびに不安が生まれる**

© 2026

Step 1：既存コードを変えずにテストだけ追加する

Copilotが既存の `app.py` を読み取り、APIの振る舞いに沿ったテストを生成：

- `conftest.py` — 一時DB生成のフィクスチャ
- `test_api.py` — 各エンドポイントの結合テスト（18件）
- `test_scenarios.py` — 複数操作の連結テスト（4件）

既存コードは **一切変更しない** — コード文脈を理解した生成がここで活きる

© 2026

22テストが全てパス — 既存コード変更なし

```
test_api.py::test_タスクを正常に作成          PASSED
test_api.py::test_タスクを正常に削除          PASSED
test_api.py::test_タスクを検索                PASSED
test_scenarios.py::test_作成_更新_完了_削除フロー PASSED
========================= 22 passed in 0.21s ====
```

既存コードの変更なしにテストカバレッジを確保できた

© 2026

Step 2：決定論的なリンターでAIの出力ブレを補う

Copilotでruff設定を生成し、以後は **ルールベースで一貫した整形** を実行：

```
[tool.ruff]
target-version = "py310"
line-length = 100

[tool.ruff.lint]
select = ["E", "F", "W", "I", "N", "UP", "B", "SIM"]
```

AIの生成は確率的で出力にばらつきがある → リンターが決定論的に統一する

© 2026

Step 3：CIで「確率論的な変更」を毎回検証する

Copilotがプロジェクトに合ったワークフローを提案：

```
# .github/workflows/ci.yml
jobs:
  lint:      # ruff check + format check
  test:      # pytest（lintパス後に実行）
```

AIが生成したコードも人が書いたコードも、同じ基準で自動検証される

この仕組みがあるからこそ、AIを安心して活用できる

© 2026

Step 4：指示書でCopilotの生成精度を底上げする

```
# Copilot Instructions
## プロジェクト概要
Flaskベースのタスク API です。
## コーディング規約
- ruffの設定に従うこと
- テスト関数名は日本語で記述してよい
## 開発時の注意事項
- エンドポイント追加時はtest_api.pyにテスト追加
- コミット前にruff check .を実行
```

確率論的なAIに文脈を与えることで、生成の的中率が上がる

© 2026

4ステップで品質基盤が整った

項目	Before	After
テスト	なし	22テスト（結合＋連結）
リンター	なし	ruff（lint + format）
CI/CD	なし	GitHub Actions
Copilot指示書	なし	copilot-instructions.md
品質管理	手動	自動化

© 2026

Tips: CopilotはPR文も自動生成できる

1. PRテンプレートと指示書を用意しておく
2. 開発完了後にCopilotへ一言依頼する

> 「このブランチの変更内容を元に、PRテンプレートに沿ったPR文を作成して」

Copilotが差分を読み取り、概要・変更内容・テスト項目を自動で埋める

© 2026

Tips: ブランチ保護もコードで管理すれば変更履歴が残る

1. `branch-protection.json` に保護ルールをJSONで定義
2. mainマージ時にGitHub Actionsが自動適用
3. ルール変更はJSON編集 → PR → マージだけ

PRレビュー必須 / ステータスチェック必須 / Force Push禁止 / 管理者にも適用

© 2026

Tips: 契約プログラミングで関数の仕様を明示しAI精度を上げる

```
@contract(
    pre(lambda task: task is not None),
    post(lambda result: "id" in result),
    pure(),
)
def task_to_dict(task):
    return {"id": task["id"], "title": task["title"], ...}
```

- `pre()` / `post()` / `pure()` で関数の意図をコードで宣言
- Copilotが仕様を正確に理解し、生成コードの品質が向上

© 2026

Tips: サプライチェーン攻撃を3層で防ぐ

GitHub Actions SHA pinning — ActionをコミットSHAで固定

```
- uses: actions/checkout@34e1148...  # v4
```

uv exclude-newer — 公開3日未満のパッケージを除外

Dependabot + cooldown — 依存更新自動化 + 3日の検疫期間

© 2026

Tips: AIエージェントはセキュアなサンドボックスで動かす

```yaml
# compose.yaml
services:
  sandbox:
    read_only: true        # root FSは読み取り専用
    network_mode: none     # オフライン既定
    cap_drop: [ALL]        # 全権限をdrop
```

- オフライン既定 → 依存取得時のみ `--online` で明示許可
- 書き込みは `/workspace` と `.sandbox/home` のみ
- 監査ログをhost側に記録

© 2026

まとめ：確率論的AIと決定論的ツールを組み合わせる

Copilot（確率論的） — コード文脈を理解し、テスト・設定・CIを生成
出力にばらつきはあるが、指示書で精度を底上げできる

リンター・CI（決定論的） — ルールに基づき毎回同じ基準で検証
AIの出力ブレを吸収し、品質を一定に保つ

両者の組み合わせが鍵
AIで素早く生成し、決定論的ツールで確実に検証する
このサイクルが、既存コードを壊さず段階的に改善する土台になる

© 2026

ご清聴ありがとうございました

リポジトリをクローンして各ブランチの差分を確認してみてください

質問はありますか？

© 2026